



Vault

A desktop app that catches every markdown file you'd hate to lose and commits it straight to a GitHub repository you own. No database, no account, no lock-in.

PLATFORM

macOS desktop, Electron or Tauri

STORAGE

A git repo of .md files, committed to main.

SCALE TARGET

~800 docs at launch, 5,000 ceiling

THE FOUR DECISIONS THAT SHAPE EVERYTHING BELOW

Commits go straight to main — no pull request.

Renaming a project moves its files via `git mv`, one commit.

Git reports what changed since the last indexed commit; the cache lives outside the repo.

OAuth device flow for auth, with a pasted fine-grained token as the alternative.

Full rationale in §11.

1 · Problem

AI assistants produce markdown constantly — specs, ADRs, prompts, meeting summaries, research notes. Today these end up scattered across chat scrollback, Downloads, desktop files named `Untitled-3.md`, and half-finished folders. There is no single place they live, so they are effectively lost the moment the chat window closes.

Existing tools solve the wrong half. Note apps (Bear, Notion) are great at reading and terrible at ownership — your content sits in someone's proprietary store. Knowledge tools (Obsidian) own the files but assume you'll organise a vault by hand. Neither is optimised for the actual moment of failure: **the five seconds between "this output is good" and "I've lost it."**

Product thesis

Capture must cost one keystroke, and storage must be a format that outlives the app. Everything else is secondary.

2 · Goals and non-goals

Goals

- Save a pasted markdown file in under five seconds, without leaving the app you're in.
- Every document reaches GitHub, tagged, as its own commit on `main`.
- Find any saved document in two keystrokes and a word.
- The vault stays fully usable if this app disappears.

Non-goals for v1

- Multi-user, sharing, or comments.
- Mobile app. Read on github.com if needed.
- Rich WYSIWYG editing. Markdown source plus preview only.
- Pull requests, review flows, or importing docs from other people's PRs.
- Any AI feature. The AI already happened elsewhere.

3 · The no-database constraint

This is a hard architectural rule, not a preference. There is no SQLite file, no server, no user account. **The git repository is the database.**

All per-document metadata lives in YAML frontmatter at the top of each `.md` file. On launch the app walks the vault folder, parses each file's frontmatter, and builds an **in-memory index** of projects, tags and full text. That index is a disposable cache: deleting it costs nothing but a rescan, and files edited outside the app — in Obsidian, in vim, on github.com — are picked up on the next scan.

atlas-api/rate-limiting-at-the-edge.md

+42

```
---
title: Rate limiting at the edge
project: Atlas API
tags: [spec, infra]
created: 2026-09-09T14:22:10Z
source: claude
---

# Rate limiting at the edge

---
```

Everything the sidebar shows is derived by reading these six lines across every file at launch. No index file, no sqlite, no server. Edit a file in any other editor and the app picks the change up on next scan.

COMMIT

add: rate limiting at the edge

BRANCH

main

Every field the UI collects is written into the file itself. This block is the entire data model.

Frontmatter schema

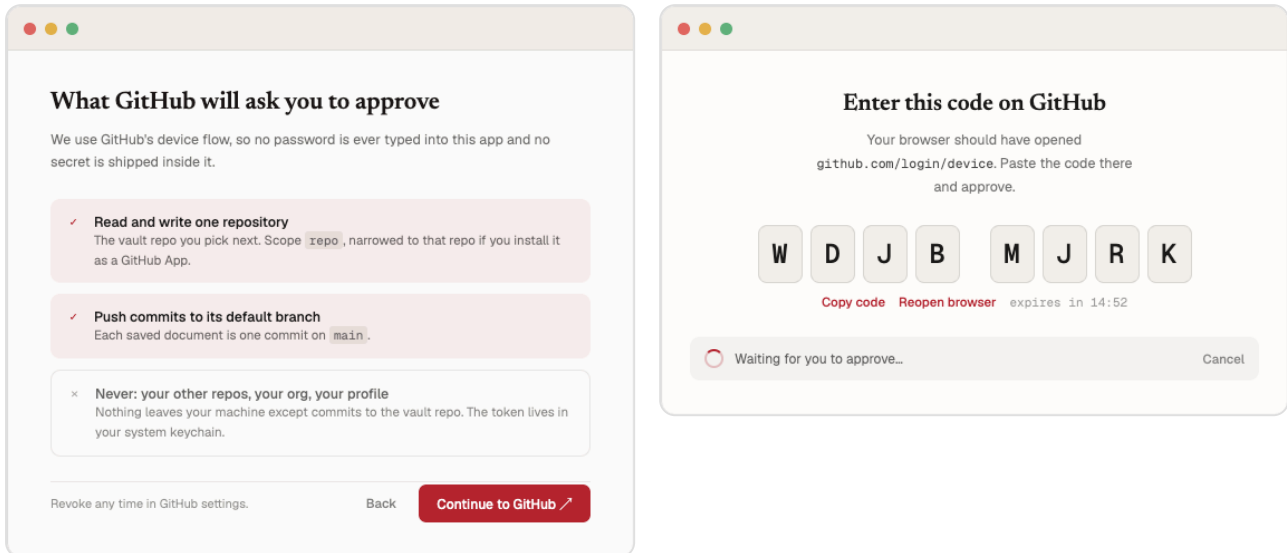
KEY	TYPE	NOTES
<code>title</code>	string	Defaults to the first # heading; editable before save.
<code>project</code>	string	Exactly one. Also determines the folder the file is written to.
<code>tags</code>	string[]	Zero or more. Free-form, autocompleted from existing tags.
<code>created</code>	ISO 8601	Set once at save. Never rewritten.
<code>source</code>	enum	<code>claude</code> · <code>chatgpt</code> · <code>github</code> · <code>manual</code> · <code>other</code> . Detected, editable.
<code>starred</code>	bool	Omitted when false, to keep files clean.

Repository layout

```
yourname/vault/  
README.md # auto-generated index, regenerated each commit  
atlas-api/ # one folder per project, slugified  
rate-limiting-at-the-edge.md  
adr-019-drop-redis.md  
research-log/  
_inbox/ # docs saved with no project  
.vault/  
views.yml # saved searches  
templates/ # frontmatter starters
```

4 · GitHub authentication

A distributed desktop binary cannot keep a client secret, so **OAuth device flow** is the only correct choice. The app never renders a GitHub login form and never embeds a webview. The access token is stored in the macOS Keychain and never written to the repo.



Left: state what we take and what we never touch, before sending the user out. Right: the eight-character device code, polled until GitHub confirms.

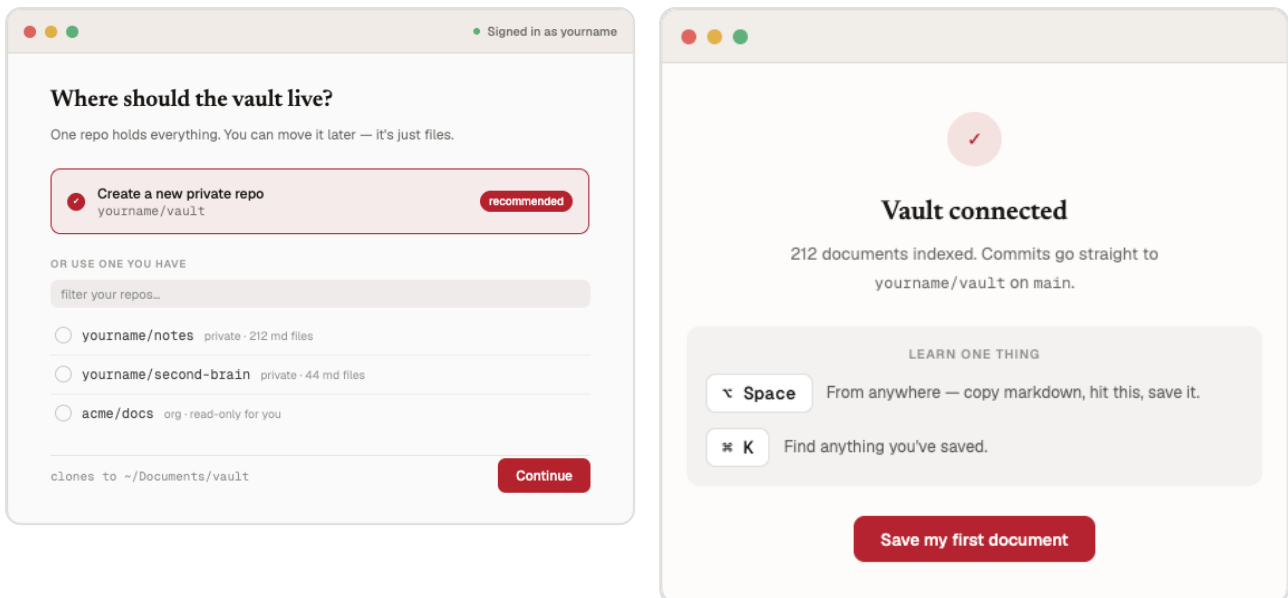
Auth sequence

1. App POSTs to `github.com/login/device/code` with its client ID and the `repo` scope.
2. GitHub returns a user code, a verification URI and a polling interval.
3. App opens the system browser to the verification URI and displays the user code.
4. App polls the token endpoint at the given interval, handling `authorization_pending`, `slow_down`, `expired_token` and `access_denied` distinctly.
5. On success, write the token to Keychain and advance to repo selection.
6. On any 401 later in the app's life, drop to a re-auth screen that reuses steps 1–5 without losing unsaved work.

Alternative to consider: shipping this as a GitHub App instead of an OAuth app narrows access to a single installed repository rather than the blanket `repo` scope. Better security story, more setup friction. Device flow with OAuth is the v1 recommendation; revisit if the vault repo ever lives in an organisation.

5 · Onboarding

Six screens, target ninety seconds. The success condition is not "account created" — it is **one document saved**.

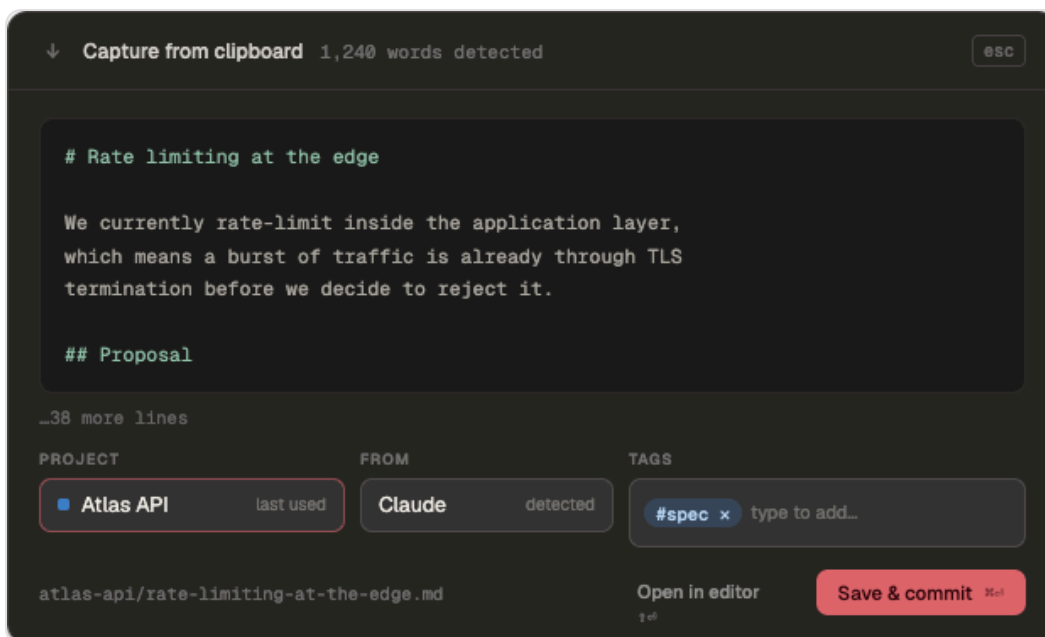


Creating a fresh private repo is the recommended path; existing repos with markdown in them are also valid and get scanned on connect. The final screen teaches exactly two shortcuts and then gets out of the way.

1. **Welcome** — one sentence of positioning, one button, plus a "start local, connect later" escape hatch.
2. **Permissions** — the plain-language scope explainer above.
3. **Device code** — code, copy button, live polling state, expiry countdown.
4. **Repo picker** — create `yourname/vault` as private, or select an existing repo; choose the local clone path.
5. **First scan** — progress bar with counts of projects, tags and files lacking frontmatter.
6. **Done** — teach `⌘Space` and `⌘K`, nothing more.

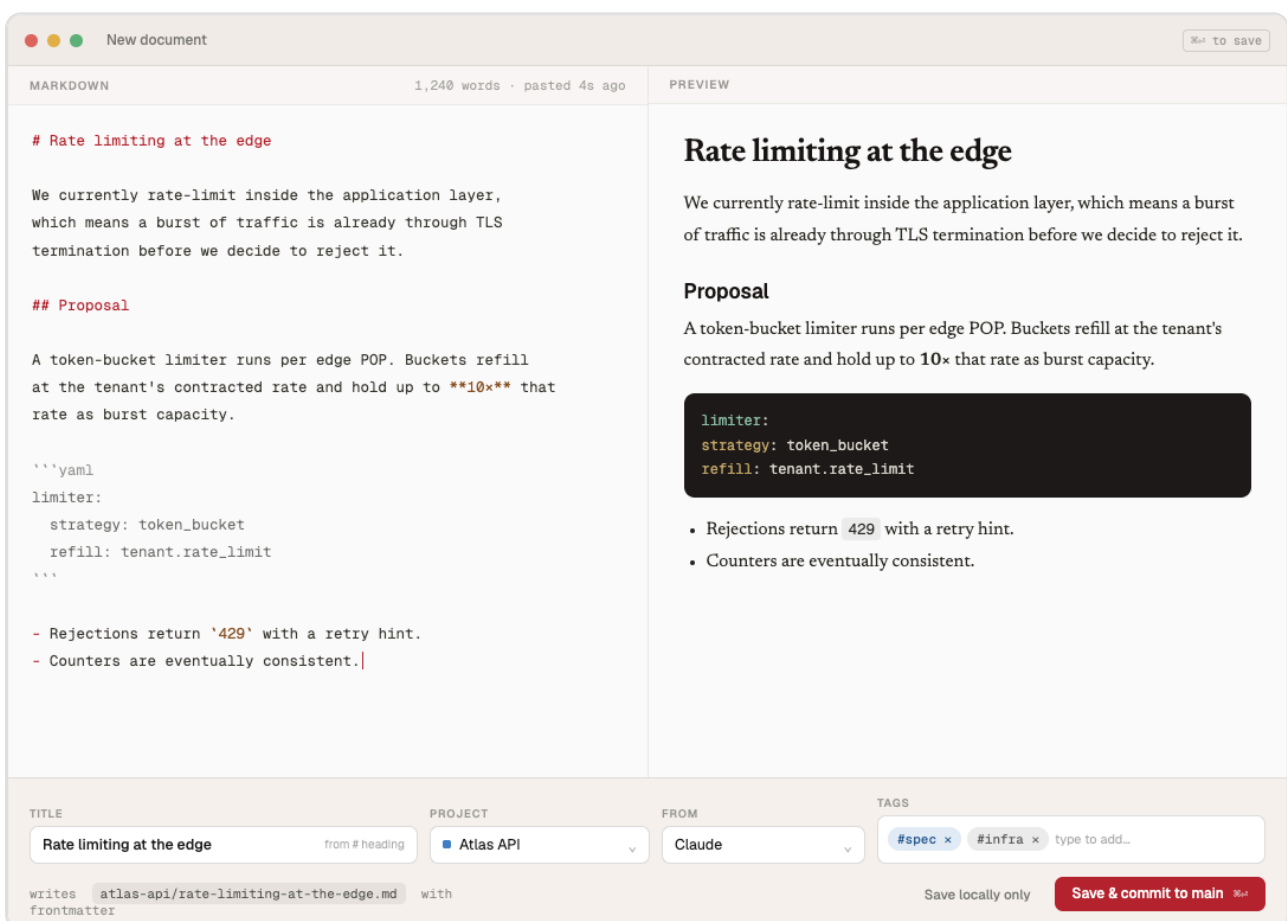
6 · Core flow: capture

The highest-frequency interaction in the product, and the reason it exists. A global hotkey opens a floating sheet over whatever application is in front. The clipboard is read automatically; project and source are pre-filled from last use and content detection. Two tabs and `⌘↵` completes it. The main window never opens.



VS Code capture sheet. Target: under five seconds from copy to commit.

For anything needing a closer look before saving, the full editor gives raw markdown on the left, live rendered preview on the right, and the metadata bar along the bottom. Mermaid fences render client-side; the file on disk stays plain text.



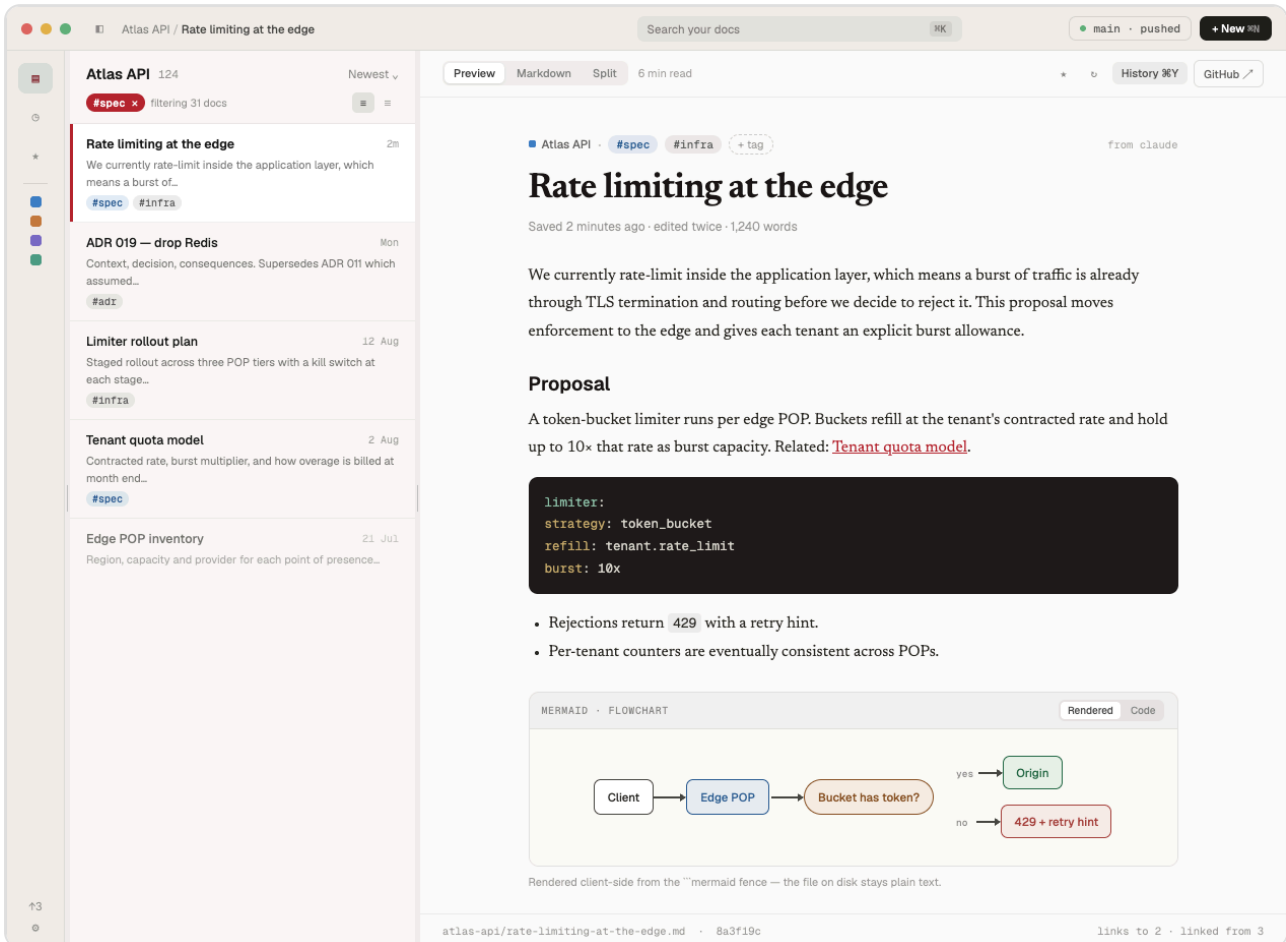
Full save window. Title is inferred from the first heading and remains editable.

What happens on save

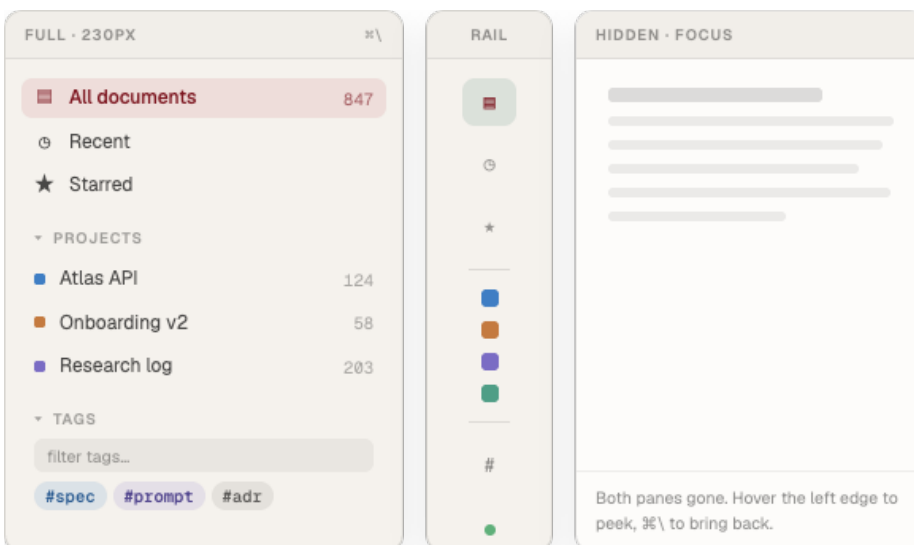
1. Compose frontmatter from the form fields; prepend to the body.
2. Write to `{project-slug}/{title-slug}.md`, de-duplicating with a numeric suffix if needed.
3. `git add` and commit with message `add: {title}`.
4. Regenerate `README.md` index; amend into the same commit.
5. Push to `main`. On failure, keep the commit local and mark the doc "not pushed" — the file is already safe on disk.
6. Update the in-memory index. No round trip required for the UI to reflect the new doc.

7 · Core flow: retrieval

Three panes: navigation, document list, reader. The sidebar cycles through three states with `⌘\` — full, icon rail, hidden — and both dividers drag to resize. Projects are the spine; tags cross-cut. At the ~800-document target the tag list needs its own filter box, which is included.

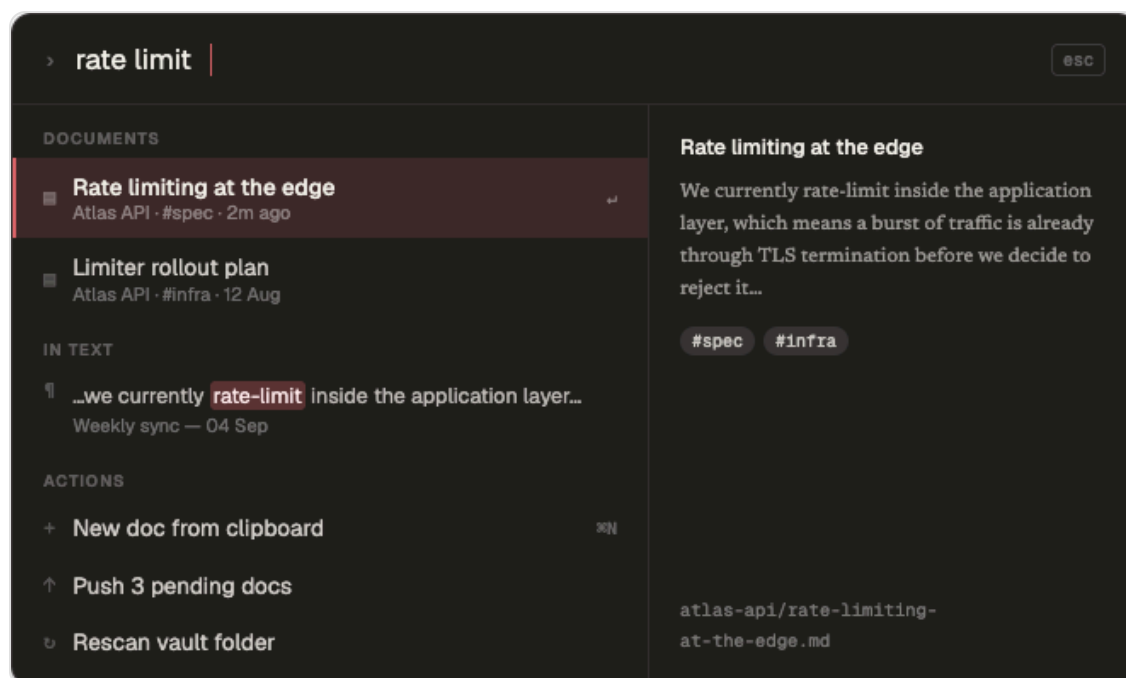


Main window with the sidebar collapsed to its icon rail, and a Mermaid diagram rendered inline in the reader.



Sidebar states. The rail keeps projects reachable by colour swatch; hidden gives a full-width reading column.

`%K` is the primary navigation surface, searching titles, frontmatter and body text against the in-memory index, with a live preview pane and an actions section for verbs.

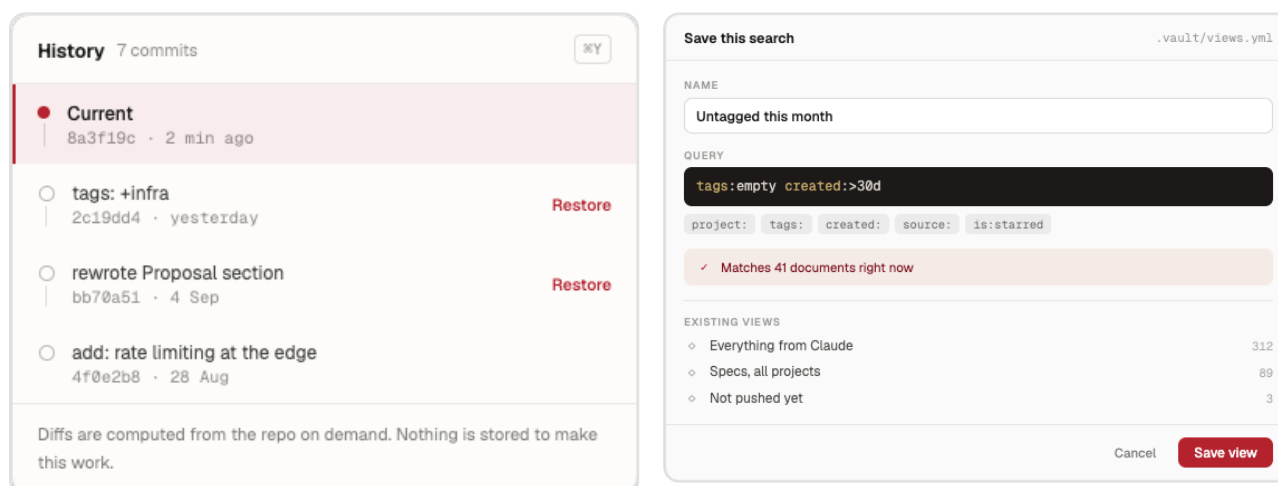


Command palette. Grouped results: documents, in-text matches, actions.

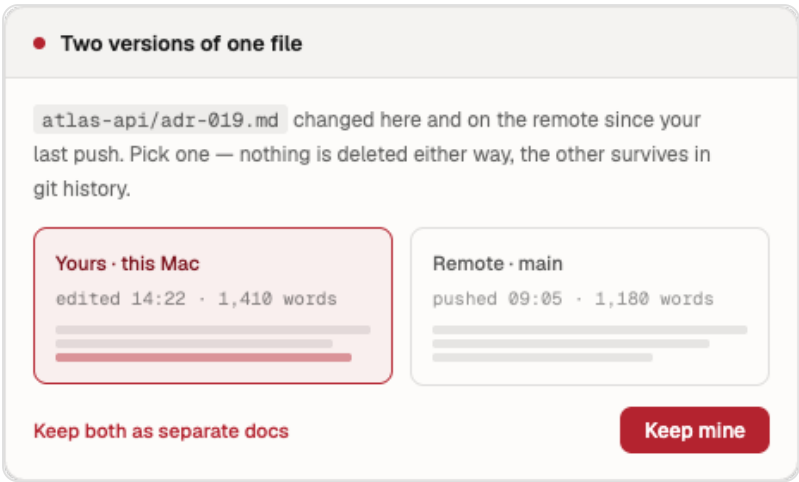
Search query grammar

OPERATOR	EXAMPLE
<code>project:</code>	<code>project:"Atlas API"</code>
<code>tags:</code>	<code>tags:spec · tags:empty</code>
<code>created:</code>	<code>created:>30d</code>
<code>source:</code>	<code>source:claude</code>
<code>is:</code>	<code>is:starred · is:unpushed</code>

8 · Supporting features



Version history reads `git log` for the file — free, because every save is already a commit. Saved searches serialise to `.vault/views.yml`.

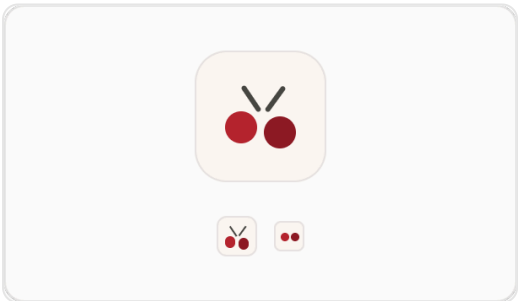


Conflict handling for two-machine use. Nothing is destroyed — the losing version stays in git history.

Feature scope, with the database test applied

FEATURE	WHERE ITS STATE LIVES	V1
Version history & restore	git log	Yes
Saved searches	.vault/views.yml	Yes
Auto-generated index README	Regenerated per commit	Yes
Templates	.vault/templates/	Yes
Wiki links & backlinks	Derived at scan, in memory	v1.1
Offline queue	Unpushed commits on disk	Yes
Semantic / vector search	Needs an embeddings store	Cut
Read counts, stale-doc nudges	Behavioural data, no home in a file	Cut

9 · Visual language



The mark is a cherry — two stones on a splayed stem — shown at app-icon, 22px and 16px. It must remain legible in the menu bar, so the small variant drops the stem.

- #b4232f accent, light surfaces
- #8f1a25 pressed, deep stone
- #e0656d accent, dark surfaces
- #5fb07a sync OK — status only
- #e4b04a not pushed — status only

Colour rule: cherry is brand, never state. Green and amber are reserved for sync status. If the accent also signalled error, the sync indicator would stop communicating.

Type: Geist for UI, Geist Mono for paths, codes and metadata, Newsreader for rendered document body — reading a saved document should feel unlike operating the app around it. Surfaces are warm off-whites (#fdfcfa, #f5f2ed) with a near-black #1e1d1b for code and overlays.

10 · Build order

milestone	ships	done when
M1 · Spine	Device-flow auth, repo clone, scanner, frontmatter parser	App lists 800 real docs from a repo
M2 · Capture	Save window, frontmatter writer, commit & push, offline queue	Paste → doc on GitHub in one PR
M3 · Retrieval	Three-pane shell, sidebar states, ⌘K, markdown + Mermaid preview	Any doc found in under three seconds
M4 · Speed	⌘Space global sheet, source detection, onboarding polish	Copy → committed in under five seconds
M5 · Durability	History drawer, conflict UI, saved searches, templates	Survives two machines and a bad merge

11 · Resolved decisions

Four questions were open at draft. All four are now closed; the rationale is recorded here because each one constrains the implementation.

D1 Commit directly to main. No pull request.

A pull request you never review is ceremony, and one with 800 commits is unusable in the GitHub web UI — it would defeat the goal of browsing the vault on github.com without the app. Direct commits give identical history, identical file browsing, and a simpler push path with fewer failure states. Nothing is lost: `git log` still powers version history, and a branch can be opened by hand any time a document genuinely warrants review. Onboarding keeps a secondary "commit to a branch" option for a future shared vault, but main is the default and the documented path.

D2 Renaming a project moves the files. One commit, using `git mv`.

The folder *is* the project. If the frontmatter said "Atlas API" while the files sat in `old-name/`, the repository would stop being self-describing — and being readable without the app is the core promise of the no-database design. The "noisy commit" concern does not survive contact with git: renames are detected by content similarity and display as moves, not as mass deletions and additions, so history follows each file intact and `git log --follow` keeps working.

Implementation: one atomic commit titled `rename project: Old → New` that both moves the files and rewrites the `project:` key. Show a confirmation naming the file count first — renaming a 200-document project should not be a stray click.

D3 Let git tell you what changed. Cache outside the repo, keyed by commit.

The insight that makes this cheap: the vault is a git repository, so it already keeps a perfect change log. Rather than stat 5,000 files on every launch, record the commit SHA that was last indexed and ask git what changed since — a single `git diff --name-status <last-sha> HEAD` returns the exact file list, at effectively zero cost, no matter how large the vault. Only those files are re-parsed.

Three-layer approach:

1. **Cold start:** full walk, but parse only the frontmatter block — stop reading each file at the closing `---`. Body text is indexed lazily in a background pass, so the UI is usable in well under a second.
2. **Warm start:** read the index cache, then apply the git diff since its recorded SHA. Typically a handful of files.
3. **Live:** a filesystem watcher on the vault folder catches edits made in other editors while the app runs, plus a manual rescan command.

The cache lives in `~/Library/Application Support/Vault/`, never in the repository, and is keyed by path plus mtime plus size so uncommitted edits are also caught. **This does not violate the no-database rule:** the cache holds nothing that is not derivable from the files. Delete it and the app rebuilds from scratch with no data loss. The test is not "does state exist anywhere" but "is the repository the only source of truth" — and it is.

D4 Stay with OAuth device flow for v1.

A GitHub App would scope access to a single installed repository, which is genuinely tighter — but it requires registering an app with a private key and, for most flows, hosting a callback endpoint. That is real infrastructure for a single-user desktop tool, and it contradicts the "no server" premise the rest of the product is built on. Device flow needs nothing hosted, ships no secret in the binary, and the token sits in the system Keychain.

Escape hatch, no engineering required: a user who wants maximum narrowness can generate a fine-grained personal access token scoped to the one vault repository and paste it instead — so support "paste a token" as an alternative on the auth screen. **Revisit the GitHub App only if** the vault moves under an organisation, or the app is ever distributed to other people.

12 · Still open

Q5 Deleting a document. Hard `git rm` — recoverable from history but invisible in the working tree — or move to a `.trash/` folder that the scanner skips? The second is friendlier and stays inside the constraint.

Q6 Push cadence. Push on every save, or commit immediately and push on a short debounce? Debouncing produces a calmer commit graph and fewer network stalls in the save path, at the cost of a slightly longer window where work is local-only.